PENSE-BÊTE · TECHNIQUE ET COMMANDES

Gestion des processus et threads sous Linux

06-210 PSE 25-27 2-5 Gestion Approfondie des Processus et Threads sous Linux — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. L'ordonnanceur distribue le temps processeur via un quantum avant une éventuelle préemption.
2. Un processus zombie a libéré ses ressources mais occupe encore une entrée dans la table des processus.
3. Les threads partagent la mémoire globale du processus tout en possédant leur propre pile d'exécution.

États des processus et cycle de vie

Notion	À retenir
États génériques	Créé (initialisé), Prêt (en file d'attente), En cours d'exécution (élu par cœur), Endormi (attente événement externe), Suspendu (pause volontaire).
Correspondance sous Linux	`R` (**TASK_RUNNING** : prêt ou exécutant), `S` (**TASK_INTERRUPTIBLE** : sommeil standard), `D` (**TASK_UNINTERRUPTIBLE** : sommeil profond matériel), `Z` (**TASK_ZOMBIE** : terminé non acquitté), `T` (**TASK_STOPPED** : suspendu).
Interruptions et signaux	Interruptions : événements **matériels** immédiats. Signaux : notifications **logicielles** asynchrones traitées au basculement noyau/utilisateur via `kill`, traités par un **handler**.
Cycle de vie détaillé	Naissance (`fork` puis `exec`), Exécution (quantum), Vie (alternance exécution/préemption/sommeil) , Mort (`exit` → état **zombie** → signal `SIGCHLD` au père).

Ordonnancement et priorités

Notion	À retenir
Principes généraux	L'ordonnanceur partage le temps processeur. Linux sépare les processus en **temps réel souple** (prioritaires) et **traditionnels**.
Politiques d'ordonnancement	Types : `SCHED_FIFO` (temps réel, priorité **0** à **99**, sans quantum), `SCHED_RR` (avec quantum), `SCHED_OTHER` / `NORMAL` (tourniquet/priorité dynamique), `SCHED_BATCH` et `SCHED_IDLE` .
Calcul priorité traditionnelle	Basé sur une **base statique**, la valeur **nice** (**-20** max à **+19** min) et un **bonus dynamique**. Priorité interne : **100** à **139** (toujours après le temps réel). Seul **root** peut baisser nice.

Changement de contexte

Notion	À retenir
Définition et causes	Processus perd l'allocation du processeur au profit d'un autre. Déclenché par : fin **quantum**, appel système **bloquant**, fin de processus ou réveil tâche **plus prioritaire**.
Mécanisme technique	`schedule` détermine le prochain processus. `context_switch` sauvegarde les **registres** et le **compteur ordinal** du sortant sur sa pile et restaure ceux de l'entrant.

Les Threads (Processus légers)

Notion	À retenir
Concept et structure	Plusieurs fils d'exécution par processus partageant l'espace mémoire global. Chaque thread a sa propre **pile d'exécution** mais partage **code**, **données**, **tas** et **fichiers ouverts**.
Avantages et inconvénients	Plus performants (création/commutation) et communication simplifiée (partage mémoire). Développement complexe : nécessite gestion de l'**atomicité** et de l'**exclusion mutuelle**.
Implémentation Linux	Utilise la librairie `pthread` . Le noyau les considère comme des **processus légers** visibles individuellement.

Commandes de gestion sous Linux

Commande	À quoi ça sert
`pstree`	affiche l'arbre généalogique des processus et les threads entre accolades
`ps -ef · ps ax`	liste tous les processus avec PID, PPID et statut détaillé
`top`	affiche en temps réel les processus gourmands en CPU et mémoire
`top -H`	visualise la consommation individuelle de chaque thread
`vmstat`	fournit des statistiques globales (interruptions, changements de contexte/s)
`kill`	envoie des signaux (ex: `SIGSTOP` pour suspendre, `SIGKILL` pour arrêt immédiat)
`nice`	lance un programme avec une priorité modifiée
`renice`	change la priorité d'un processus en cours via son PID
`ps -o`	personnalise l'affichage (classe d'ordonnancement, priorité interne)

Pièges et confusions

- ⚠ Confondre l'échelle de priorité nice (la valeur la plus basse est la plus haute) avec une échelle classique.
- ⚠ Croire qu'un processus en état D peut être tué immédiatement alors qu'il attend une opération matérielle.
- ⚠ Penser qu'un processus zombie consomme de la mémoire vive ou du CPU.
- ⚠ Oublier que les threads d'un même processus partagent le même PID mais ont des SPID différents.
- ⚠ Confondre les processus temps réel (priorité 0 à 99) avec les processus normaux (priorité 100 à 139).
- ⚠ Ignorer que seul l'utilisateur root peut diminuer la valeur nice pour augmenter la priorité.

Sigles et définitions

Processus	Programme en cours d'exécution avec allocation de ressources.
Ordonnanceur	Composant chargé de distribuer le temps processeur.
Quantum	Durée finie d'allocation du processeur à un processus.
Signal	Événement logiciel asynchrone envoyé à un processus.
Thread	Flux d'exécution léger partageant la mémoire globale.
Exclusion mutuelle	Mécanisme de gestion des accès concurrents entre threads.
Changement de contexte	Sauvegarde de l'état du processus sortant et restauration de l'entrant.

Auto-test

1. Quel appel système permet de créer un nouveau processus par duplication ?
2. Quelle commande permet de visualiser la consommation individuelle de chaque thread ?
3. Quelle est la plage de priorité interne des processus temps réel ?
4. Quelle fonction est chargée de traiter un signal reçu ?
5. Quel est l'état d'un processus terminé qui n'a pas encore été acquitté par son père ?

Réponses : **1.** fork · **2.** top -H · **3.** 0 à 99 · **4.** handler · **5.** Z (TASK_ZOMBIE)

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.